



STUDENT DETAILS

Name: Job Roman Barasa

Admission Number: BSCCS/2024/42922

Course: Bachelor of Science in Computer Science

Unit Code: BIT 4138

Unit Name: Advanced Cryptography

Lecturer Name: Mr Michael Nyoro

Academic Year: 4.2

PROJECT DETAILS

Project Title: Course Work Advanced Cryptography

Creative Tasks Done

Encrypted and decrypted words using Vigenere and Caesar Cipher

Created random number generators

Modified an already existing XOR cipher

Additional assignments week 1 to 3

Selected Technologies

Python

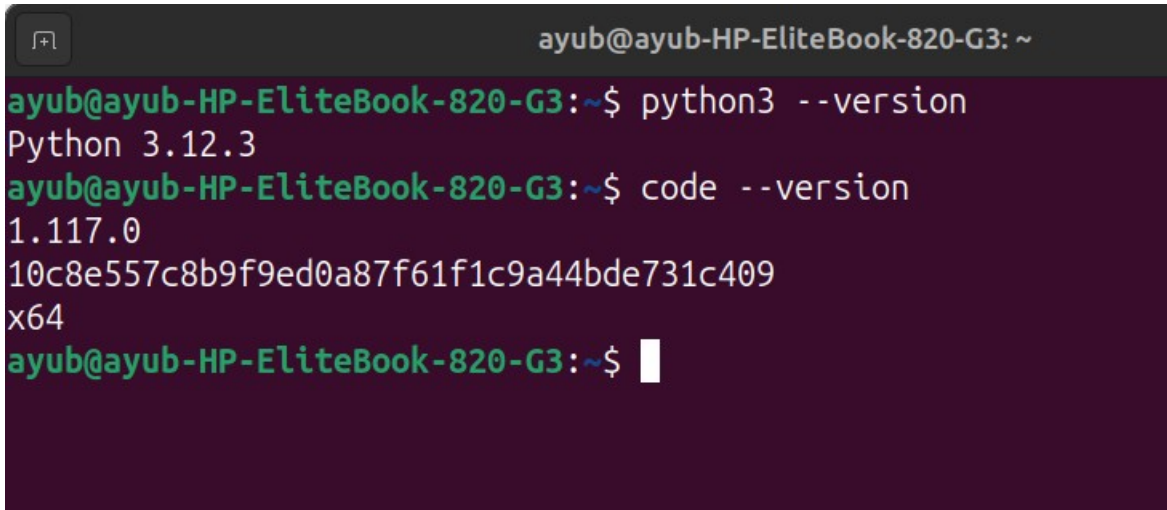
Python libraries

Github Link: [Files](#)

Week 1: Cryptography Environment Setup.....	3
Installation and Configuration of Cryptography Development Environment.....	3
Reflection.....	4
Class Activity 1: Manual Encryption.....	5
Class Activity 2: Cipher Comparison Table.....	6
Practical Task Report: Comparison of Caesar Cipher and Vigenère Cipher.....	7
Extra Practical Task.....	8
Week 2: Classical Cryptography and Cipher Design.....	9
Implementation of Classical Encryption Algorithms.....	9
Reflection.....	11
Practical Demonstrations.....	12
Class Activities.....	14
Advanced Practice Tasks.....	16
Week 3: Stream Ciphers and Randomness Testing.....	18
Implementation of Stream Cipher Systems and Random Sequence Analysis..	18
Reflection.....	19
Practical Demonstrations.....	20
Assignment 1.....	23
Assignment 2.....	24
Week 4: STREAM CIPHER CRYPTANALYSIS.....	26
Reflection.....	28
Week 5: Public Key Cryptography.....	29
Reflection.....	30

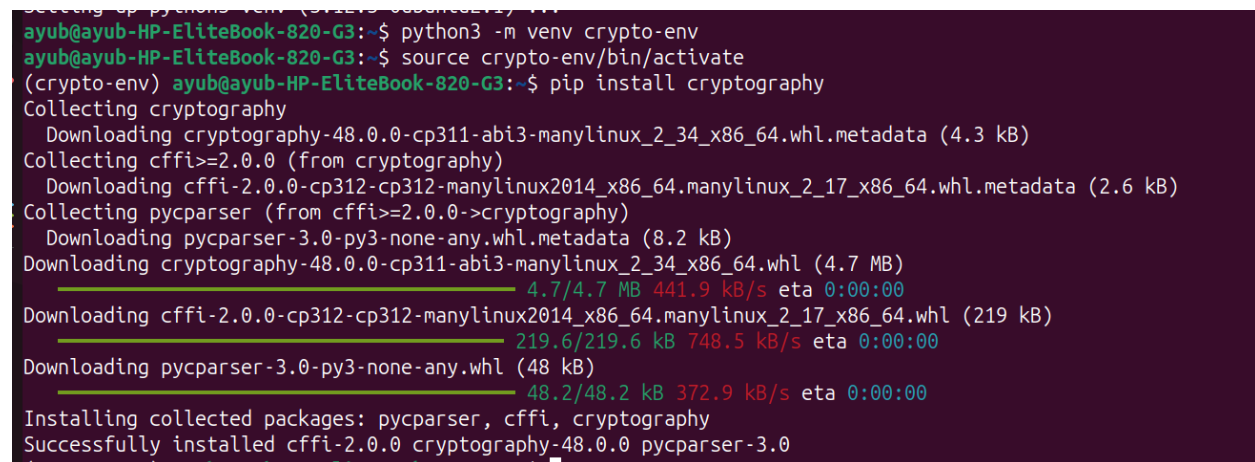
Week 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

A terminal window with a dark background and green text. The prompt is 'ayub@ayub-HP-EliteBook-820-G3: ~'. The user enters 'python3 --version' and the output is 'Python 3.12.3'. Then the user enters 'code --version' and the output is '1.117.0', followed by a long alphanumeric string '10c8e557c8b9f9ed0a87f61f1c9a44bde731c409' and 'x64'. The prompt returns to 'ayub@ayub-HP-EliteBook-820-G3: ~\$' with a cursor.

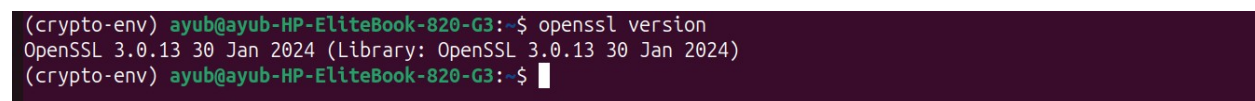
```
ayub@ayub-HP-EliteBook-820-G3: ~$ python3 --version
Python 3.12.3
ayub@ayub-HP-EliteBook-820-G3: ~$ code --version
1.117.0
10c8e557c8b9f9ed0a87f61f1c9a44bde731c409
x64
ayub@ayub-HP-EliteBook-820-G3: ~$
```

Figure 1: Installation of Python and VS Code

A terminal window showing the setup of a virtual environment and installation of cryptography libraries. The prompt is '(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~\$'. The user enters 'python3 -m venv crypto-env', then 'source crypto-env/bin/activate', and finally 'pip install cryptography'. The output shows the collection and downloading of cryptography-48.0.0, cffi-2.0.0, and pycparser-3.0. Progress bars are shown for the downloads. The final output is 'Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0'.

```
ayub@ayub-HP-EliteBook-820-G3: ~$ python3 -m venv crypto-env
ayub@ayub-HP-EliteBook-820-G3: ~$ source crypto-env/bin/activate
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$ pip install cryptography
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-48.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
  4.7/4.7 MB 441.9 kB/s eta 0:00:00
Downloading cffi-2.0.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (219 kB)
  219.6/219.6 kB 748.5 kB/s eta 0:00:00
Downloading pycparser-3.0-py3-none-any.whl (48 kB)
  48.2/48.2 kB 372.9 kB/s eta 0:00:00
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0
```

Figure 2: Installation of Cryptography Libraries

A terminal window showing the OpenSSL version check. The prompt is '(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~\$'. The user enters 'openssl version' and the output is 'OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)'. The prompt returns to '(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~\$' with a cursor.

```
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
(crypto-env) ayub@ayub-HP-EliteBook-820-G3: ~$
```

Figure 3: OpenSSL Installation Test

```
(crypto-env) ayub@ayub-HP-EliteBook-820-G3:~$ nano encrypt.py
(crypto-env) ayub@ayub-HP-EliteBook-820-G3:~$ python3 encrypt.py
Original: Hello World
Encrypted: qAAAAABqJWXYBFNZKrK0Xdqxf2kTk anuzCtj Njlicqa8ja8XP7rInS3v3Fy86V7z5VZaLinE018nVe5t0h3ZDKdnMfcd0K0q=
```

Figure 4: Local Encryption Script Execution

```
ayub@ayub-HP-EliteBook-820-G3:~$ mkdir CryptoProject
cd CryptoProject
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/ayub/CryptoProject/.git/
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ echo "# CryptoProject" > README.md
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git add .
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git commit -m "Initial commit"
[master (root-commit) d3b4b95] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$ git log --oneline
d3b4b95 (HEAD -> master) Initial commit
ayub@ayub-HP-EliteBook-820-G3:~/CryptoProject$
```

Figure 5: GitHub Repository Initialization

Reflection

I learned how to set up a Python cryptography environment, configure OpenSSL and Git, and execute encryption scripts. The main challenge was resolving the pip installation restriction in Ubuntu. The screenshots demonstrate successful installation, configuration of security tools, encryption testing, and project repository initialization.

Class Activity 1: Manual Encryption

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

S- W

E- I

C- G

U- Y

R- V

I- M

T- X

Y- C

SECURITY using Caesar Cipher becomes- WIGYVMXC

Figure 6: Encrypt "SECURITY" using Caesar Cipher with shift 4.

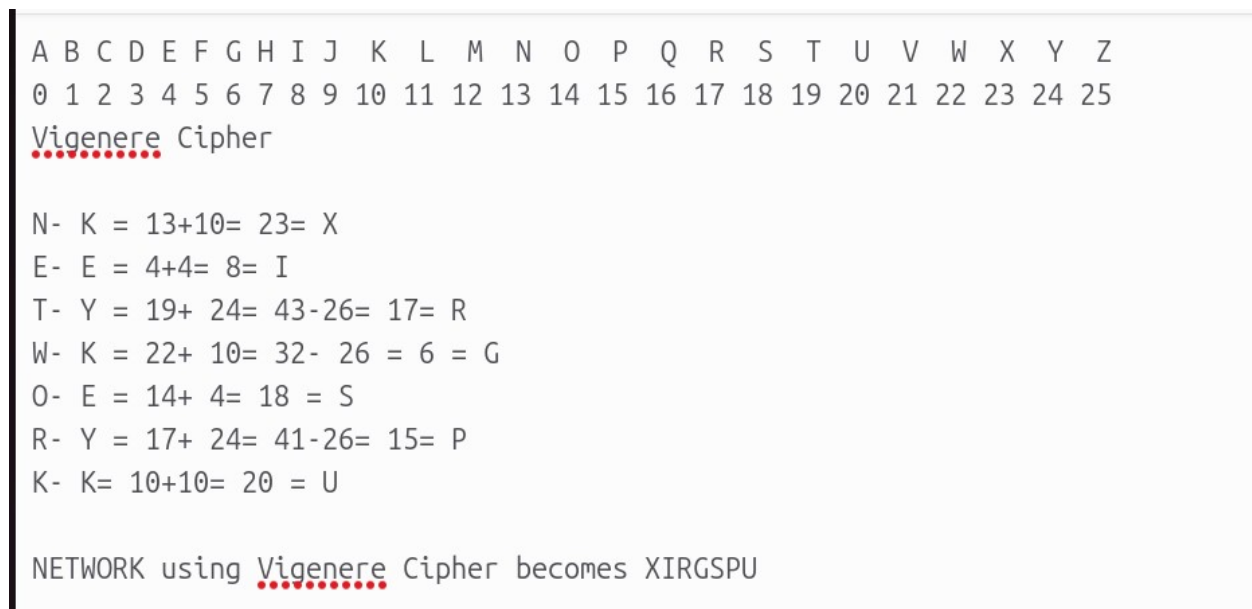


Figure 7: Encrypt “NETWORK” using Vigenère Cipher using keyword “KEY”.

Class Activity 2: Cipher Comparison Table

	Caesar Cipher	Vigenere Cipher
Encryption Method	Substitutes each letter by shifting it a fixed number of positions in the alphabet.	Multiple Caesar shifts based on a keyword.
Key Usage	A single numerical shift value ie. 4	A keyword that determines different shifts for different letters.
Security Level	Easy to crack using brute force or frequency analysis.	Higher security than Caesar Cipher as it uses multiple shifts.
Advantages	Simple to understand and implement	More secure than Caesar Cipher

	Caesar Cipher	Vigenere Cipher
	Requires little computation	Harder to break without knowing the keyword.
Weaknesses	Very easy to decrypt due to only 25 possible shifts.	Vulnerable to cryptanalysis if the keyword is short or repeated.
Ease Of Implementation	Very easy, can be done manually with minimal effort.	Moderately easy. Requires matching the keyword with the plaintext and performing multiple shifts

Practical Task Report: Comparison of Caesar Cipher and Vigenère Cipher

Introduction

Cryptography is the study of securing information by converting it into unreadable formats. Two basic classical encryption techniques are the Caesar Cipher and the Vigenère Cipher. Both are substitution ciphers used to understand basic encryption concepts.

Encryption Process

The Caesar Cipher works by shifting each letter in the plaintext by a fixed number of positions in the alphabet. For example, a shift of 4 changes A → E.

The Vigenère Cipher uses a keyword to determine different shift values for each letter. Each letter of the keyword defines a different Caesar shift, making the encryption more complex.

Advantages and Disadvantages

Caesar Cipher is simple and easy to implement but very weak in security.

Vigenère Cipher is more secure and harder to break, but it is more complex and still vulnerable if the keyword is short or reused.

Security Comparison

Caesar Cipher has very low security because it can be easily broken using brute force (only 25 possible shifts).

Vigenère Cipher is stronger because it uses multiple shifts, making frequency analysis more difficult, however it can still be cracked with modern cryptanalysis.

Modern Relevance

Caesar Cipher is mainly used for educational purposes only.

Vigenère Cipher is also mostly educational today, but it introduced ideas that influenced modern encryption techniques.

Conclusion

Both ciphers are important in learning cryptography basics. However, neither is secure for modern communication. Vigenère Cipher is stronger than Caesar Cipher but still not suitable for real-world security systems.

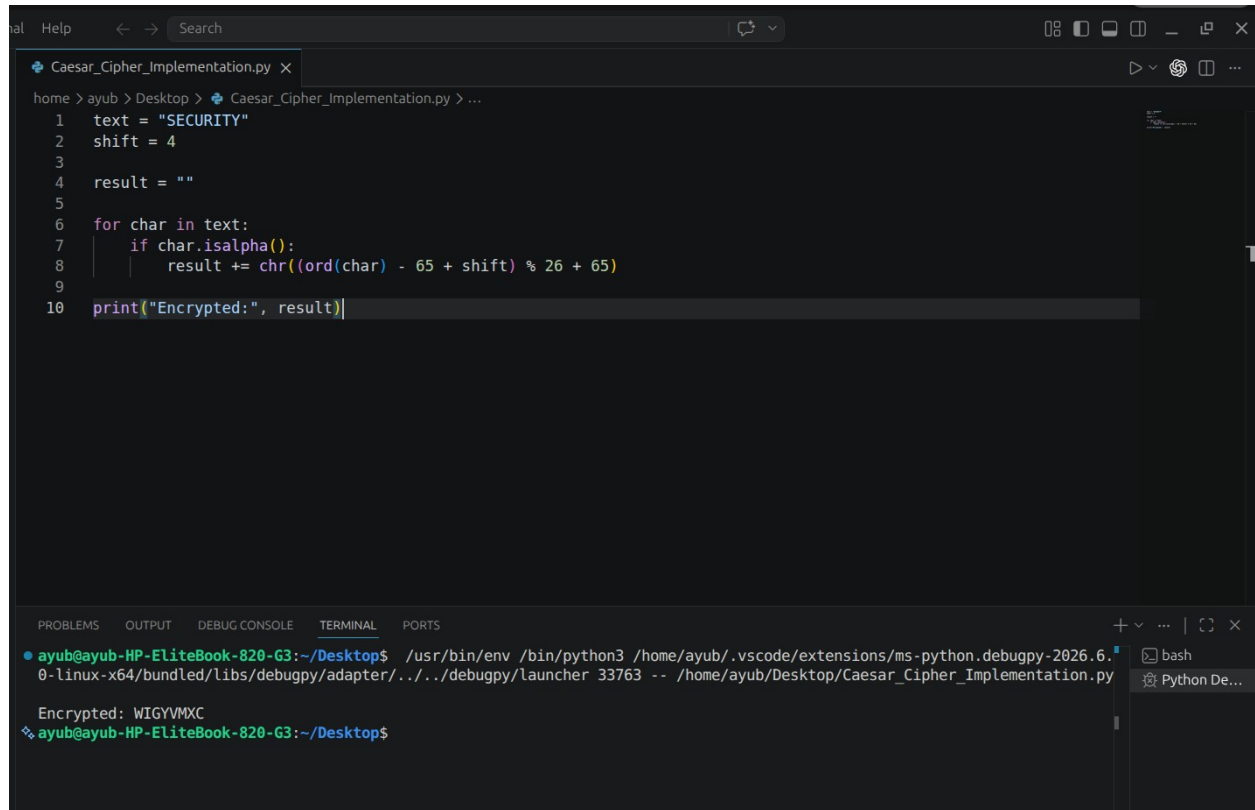
Extra Practical Task

The screenshot shows the 'Classical Cipher Studio' website. The header includes the URL '8gwifi.org' and navigation links: 'Tutorials', 'URL Shortener', 'JSON Beautifier', 'Temp Email', 'Hire Me!', and a 'Buy me a coffee' button. The main heading is 'Classical Cipher Studio' with a subtitle 'Type once – see many ciphers at a glance. Toggle Encode/Decode to switch direction.' The interface is divided into three main sections: 'Input & Mode', 'Parameters', and 'Outputs'. The 'Input & Mode' section has a 'Decode mode' checkbox and a text input field containing 'NETWORK'. The 'Parameters' section contains settings for 'Caesar Shift' (4), 'Affine A' (1), 'Affine B' (8), 'Vigenère Key' (KEY), 'Scytale Columns' (5), and 'Rail Fence Rails' (3). The 'Outputs' section displays the results for various ciphers: Caesar (shift 4) with output 'RIXASVO', ROT13 with 'ARGJBEX', ROT18 with 'ARGJBEX', ROT47 with '}t% (~#z', Affine (a=1, b=8) with 'VMBEWZS', Enigma (M3) with 'OZARWIY', Rail Fence (rails=3) with 'NOEWRTK', ROT5 (0-9) with 'NETWORK', Atbash with 'MVGDLIP', Vigenère (KEY) with 'XIRGSPU', and Scytale (cols=5) with 'NREKTWO'. Each output has a 'Copy' button.

Figure 8: Research and test one free online classical cipher simulator

Week 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

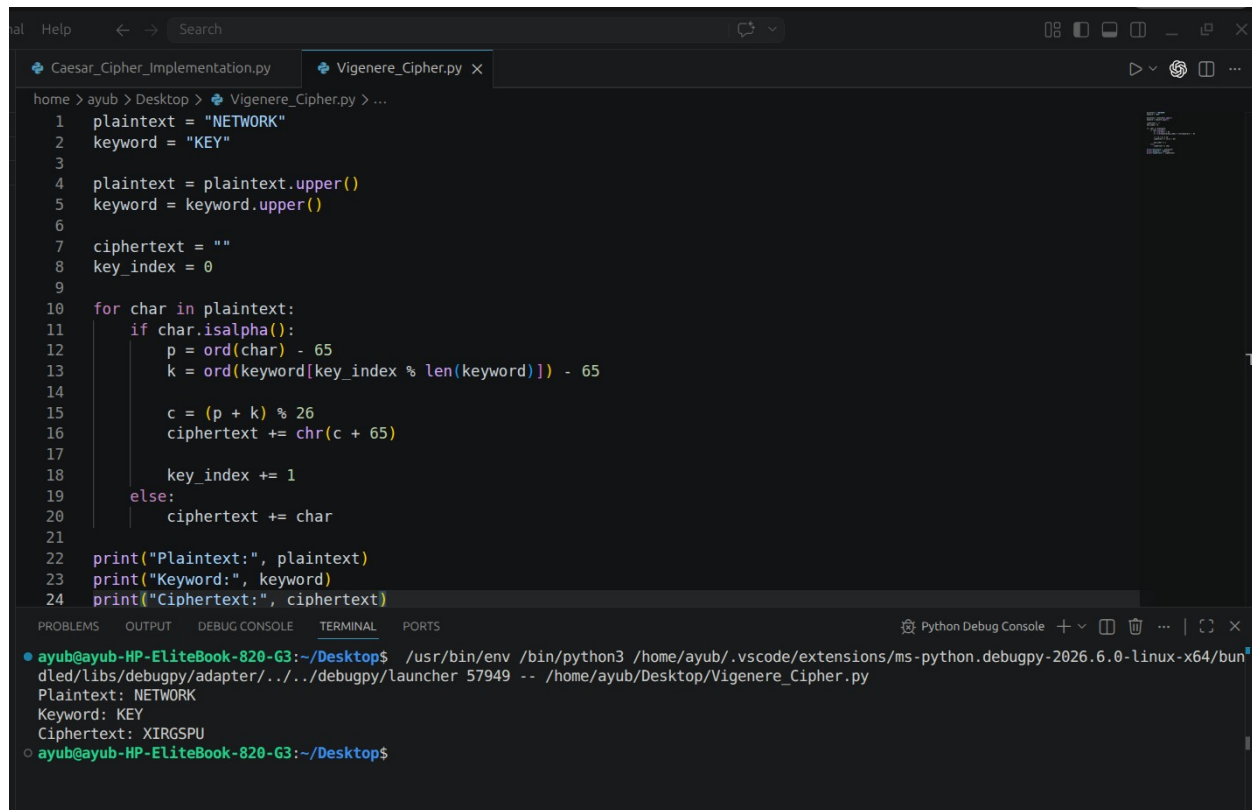


```
Caesar_Cipher_Implementation.py x
home > ayub > Desktop > Caesar_Cipher_Implementation.py > ...
1  text = "SECURITY"
2  shift = 4
3
4  result = ""
5
6  for char in text:
7      if char.isalpha():
8          result += chr((ord(char) - 65 + shift) % 26 + 65)
9
10 print("Encrypted:", result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 33763 -- /home/ayub/Desktop/Caesar_Cipher_Implementation.py
Encrypted: WIGYVMXC
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 9: Caesar Cipher Implementation

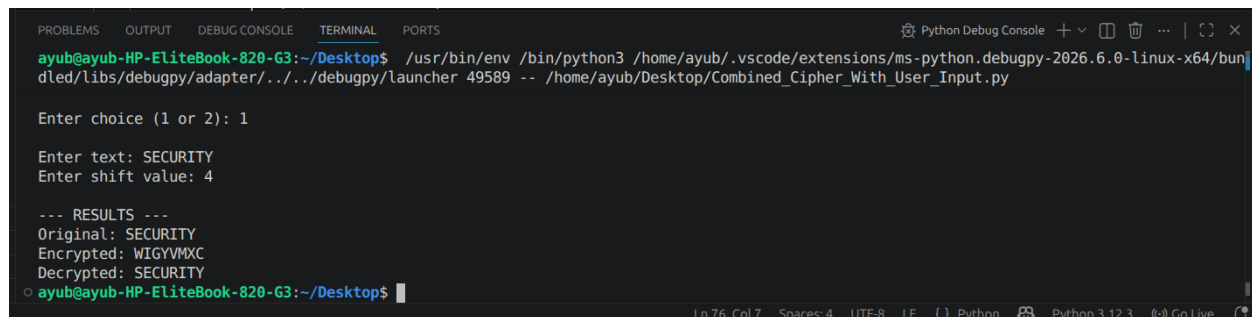


```
home > ayub > Desktop > Vigenere_Cipher.py > ...
1  plaintext = "NETWORK"
2  keyword = "KEY"
3
4  plaintext = plaintext.upper()
5  keyword = keyword.upper()
6
7  ciphertext = ""
8  key_index = 0
9
10 for char in plaintext:
11     if char.isalpha():
12         p = ord(char) - 65
13         k = ord(keyword[key_index % len(keyword)]) - 65
14
15         c = (p + k) % 26
16         ciphertext += chr(c + 65)
17
18         key_index += 1
19     else:
20         ciphertext += char
21
22 print("Plaintext:", plaintext)
23 print("Keyword:", keyword)
24 print("Ciphertext:", ciphertext)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 57949 -- /home/ayub/Desktop/Vigenere_Cipher.py
Plaintext: NETWORK
Keyword: KEY
Ciphertext: XIRGSPU
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 10: Vigenere Cipher Implementation



```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 49589 -- /home/ayub/Desktop/Combined_Cipher_With_User_Input.py

Enter choice (1 or 2): 1

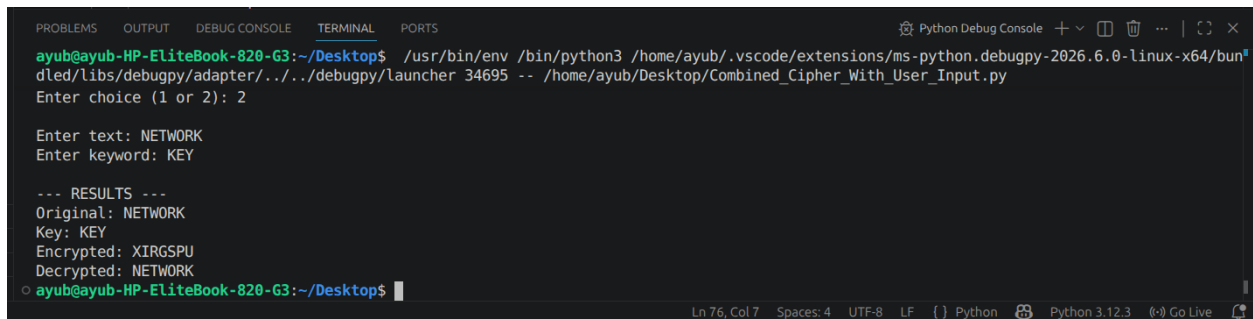
Enter text: SECURITY
Enter shift value: 4

--- RESULTS ---
Original: SECURITY
Encrypted: WIGYVMXC
Decrypted: SECURITY
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Ln 76, Col 7 Spaces: 4 UTF-8 LF Python Python 3.12.3 Go Live

Figure 11: Caesar Cipher User Input Validation Interface

Figure 12: Caesar Cipher User Input Validation Interface



```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 34695 -- /home/ayub/Desktop/Combined_Cipher_With_User_Input.py
Enter choice (1 or 2): 2

Enter text: NETWORK
Enter keyword: KEY

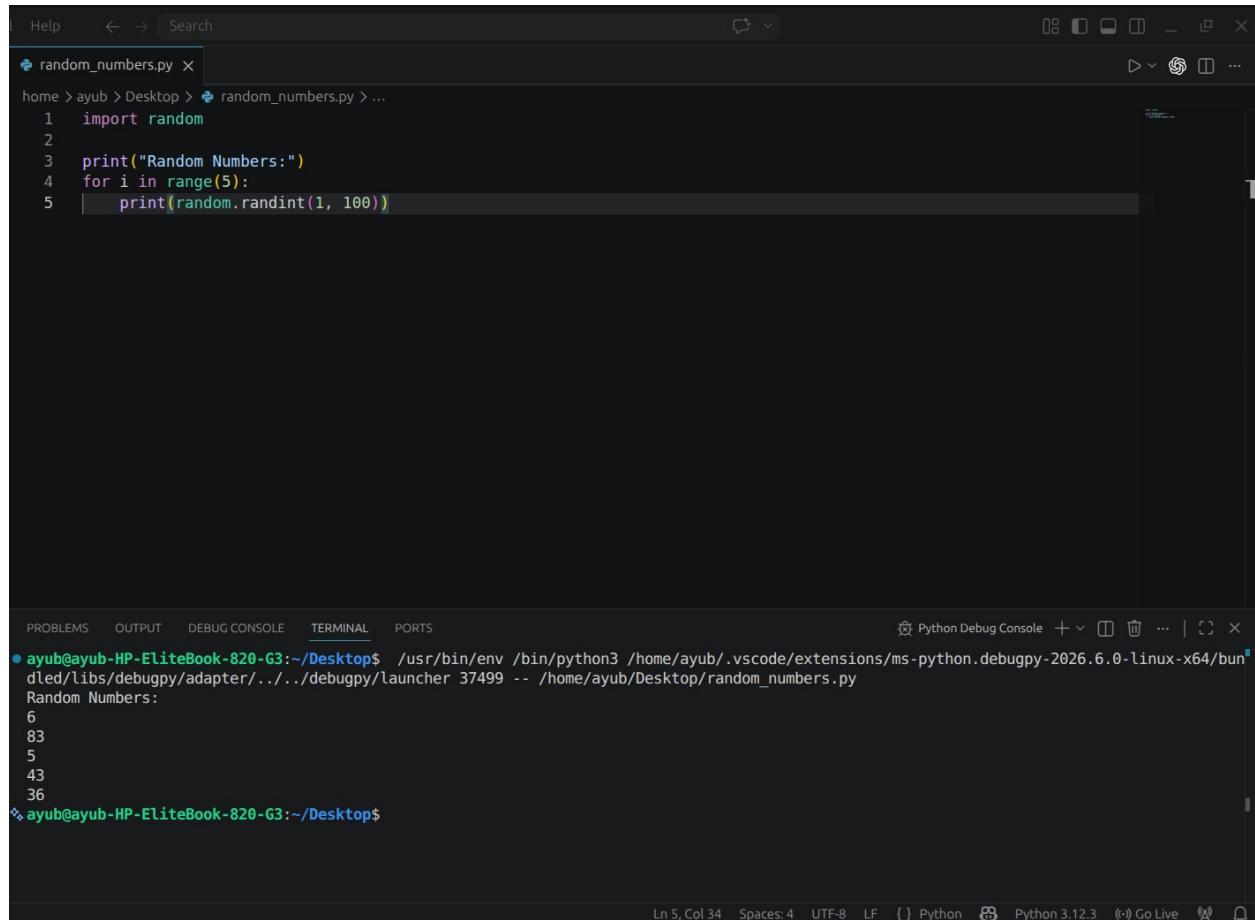
--- RESULTS ---
Original: NETWORK
Key: KEY
Encrypted: XIRGSPU
Decrypted: NETWORK
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 13: Vigenere Cipher User Interface Validation

Reflection

I learned that encryption algorithms convert readable text into ciphertext using mathematical rules and keys. Keys are important because they control the encryption and decryption process. The encryption workflow involves plaintext, a key, encryption, and decryption. I identified weaknesses in classical ciphers, including susceptibility to brute-force and frequency analysis attacks.

Practical Demonstrations



The image shows a Visual Studio Code editor window with a Python file named `random_numbers.py` open. The code in the file is as follows:

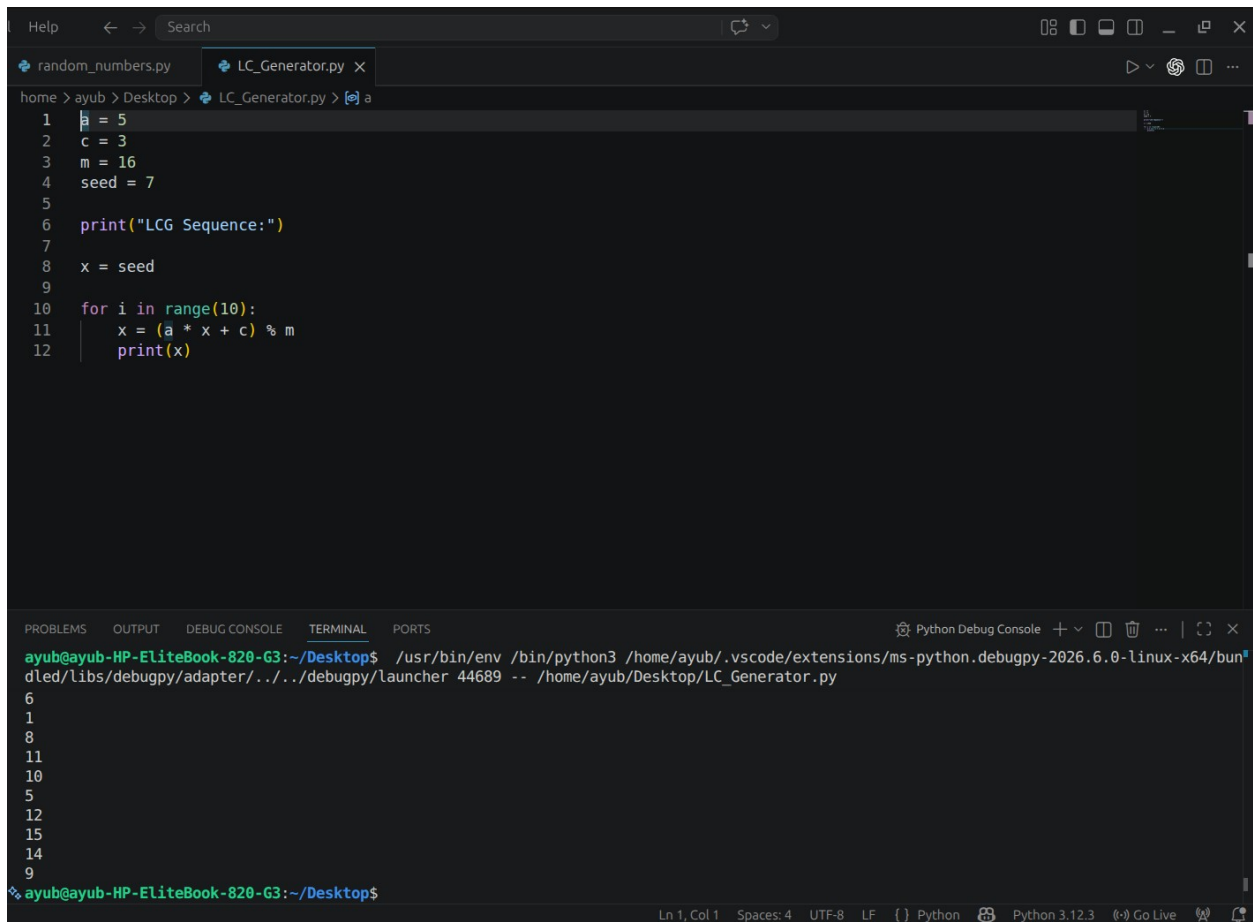
```
1 import random
2
3 print("Random Numbers:")
4 for i in range(5):
5     print(random.randint(1, 100))
```

Below the editor, the TERMINAL panel is active, showing the command used to run the script and its output:

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 37499 -- /home/ayub/Desktop/random_numbers.py
Random Numbers:
6
83
5
43
36
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

The status bar at the bottom indicates the current cursor position is at line 5, column 34, with 4 spaces. The file encoding is UTF-8, line endings are LF, and the file is a Python script using Python 3.12.3.

Figure 14: Python Random Numbers



The screenshot shows a VS Code editor with a file named `LC_Generator.py` open. The code implements a Linear Congruential Generator (LCG) with parameters $a=5$, $c=3$, $m=16$, and $\text{seed}=7$. It prints the LCG sequence for the first 10 iterations. The terminal output shows the sequence: 6, 1, 8, 11, 10, 5, 12, 15, 14, 9.

```
1 a = 5
2 c = 3
3 m = 16
4 seed = 7
5
6 print("LCG Sequence:")
7
8 x = seed
9
10 for i in range(10):
11     x = (a * x + c) % m
12     print(x)
```

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 44689 -- /home/ayub/Desktop/LC_Generator.py
6
1
8
11
10
5
12
15
14
9
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 15: Linear Congruential Generator (LCG)



The screenshot shows a VS Code editor with a file named `XOR_Cipher.py` open. The code implements an XOR Stream Cipher. It takes the plaintext "HELLO" and a key of 121, encrypts it to the list [49, 60, 53, 53, 54], and then decrypts it back to "HELLO".

```
1 plaintext = "HELLO"
2 key = 121
3
4 encrypted = []
5
6 for char in plaintext:
7     encrypted.append(ord(char) ^ key)
8
9 print("Encrypted:", encrypted)
10
11 decrypted = ""
12
13 for value in encrypted:
14     decrypted += chr(value ^ key)
15
16 print("Decrypted:", decrypted)
```

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 44623 -- /home/ayub/Desktop/XOR_Cipher.py
Encrypted: [49, 60, 53, 53, 54]
Decrypted: HELLO
```

Figure 16: XOR Stream Cipher

```
home > ayub > Desktop > OTP_Simulation.py > ...
1 import random
2
3 plaintext = "HELLO"
4
5 key = [random.randint(0, 255) for _ in plaintext]
6
7 encrypted = []
8
9 for i in range(len(plaintext)):
10     encrypted.append(ord(plaintext[i]) ^ key[i])
11
12 print("Plaintext:", plaintext)
13 print("Key:", key)
14 print("Encrypted:", encrypted)
15
16 decrypted = ""
17
18 for i in range(len(encrypted)):
19     decrypted += chr(encrypted[i] ^ key[i])
20
21 print("Decrypted:", decrypted)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapters/.../
py/launcher 59835 -- /home/ayub/Desktop/OTP_Simulation.py
Plaintext: HELLO
Key: [185, 74, 97, 94, 62]
Encrypted: [241, 15, 45, 18, 113]
Decrypted: HELLO
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 17: OTP Simulation

Class Activities

Assignment 1

```
home > ayub > Desktop > Pseudorandom_Number_Generator.py > ...
1 # Pseudorandom Number Generator
2
3 seed = int(input("Enter seed value: "))
4
5 a = 5 # multiplier
6 c = 3 # increment
7 m = 16 # modulus
8
9 x = seed
10
11 print("\nGenerated Sequence:")
12
13 for i in range(20):
14     x = (a * x + c) % m
15     print(f"Value {i+1}: {x}")
16
17 print("""
18 The generated sequence appears random initially but eventually repeats. The number of values generated before repetition is called the period. In this exper
19 """)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapters/.../debug
py/launcher 42325 -- /home/ayub/Desktop/Pseudorandom_Number_Generator.py
Value 15: 13
Value 16: 4
Value 17: 7
Value 18: 6
Value 19: 1
Value 20: 8

The generated sequence appears random initially but eventually repeats. The number of values generated before repetition is called the period. In this experiment, the se
quence repeated after cycling through the available values determined by the modulus. Larger modulus values generally improve randomness and increase the period length.
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 18: pseudorandom sequence generator in Python.

Assignment 2

One-Time Pad (OTP), RC4 Stream Cipher, and Real-World Applications

Introduction

Stream ciphers encrypt data one bit or one byte at a time using a keystream. Two well-known examples are the One-Time Pad (OTP) and RC4.

One Time Pad

The One-Time Pad is an encryption technique that uses a truly random key equal in length to the message. Each character of the plaintext is combined with the corresponding key character to produce ciphertext

Advantages

Provides perfect secrecy when used correctly.

Extremely resistant to cryptanalysis.

Disadvantages

Requires a key as long as the message.

Secure key distribution is difficult.

Keys must never be reused.

RC4 Stream Cipher

RC4 is a stream cipher developed by Ron Rivest. It generates a pseudorandom keystream that is combined with plaintext using XOR operations.

Advantages

Simple and fast implementation.

Low computational requirements.

Disadvantages

Vulnerable to several cryptographic attacks.

No longer recommended for secure communications.

Real World Applications

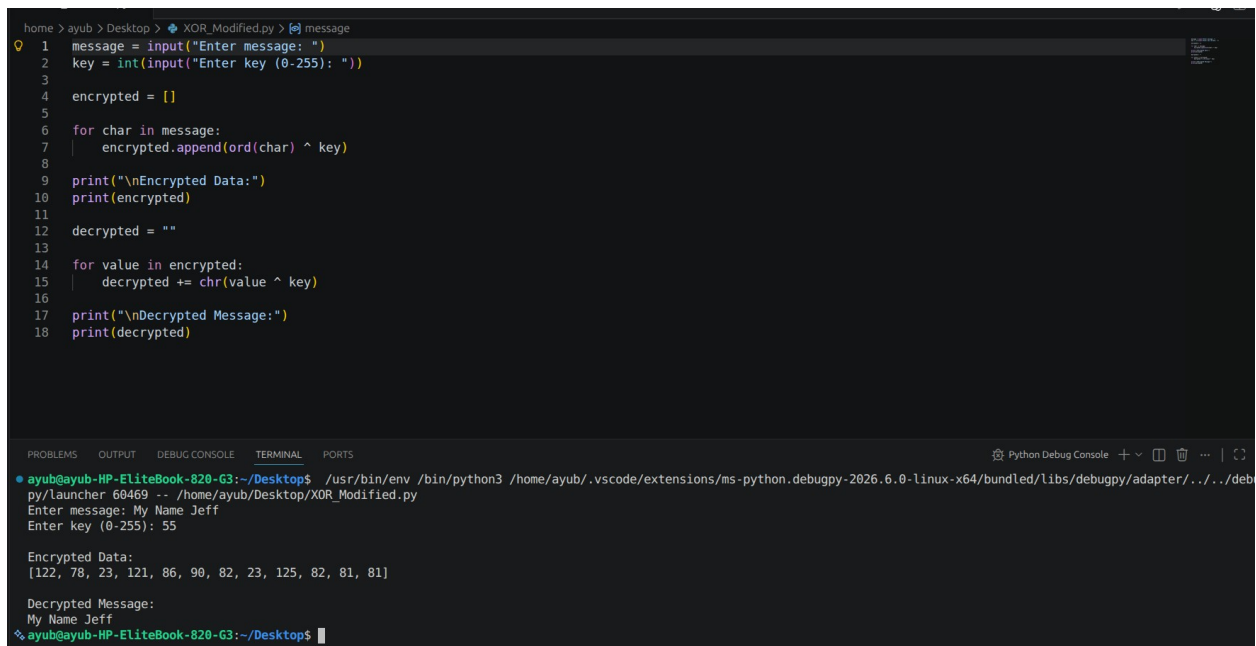
One-Time Pads have historically been used for highly sensitive military and diplomatic communications. RC4 was widely used in wireless security protocols and internet communications but has largely been replaced by more secure algorithms such as AES-based encryption.

Conclusion

OTP offers theoretically perfect security but is impractical for most applications. RC4 was efficient and popular but is now considered insecure. Modern systems rely on stronger cryptographic algorithms that provide both security and practicality.

Advanced Practice Tasks

Task 1



```
home > ayub > Desktop > XOR_Modified.py > message
1 message = input("Enter message: ")
2 key = int(input("Enter key (0-255): "))
3
4 encrypted = []
5
6 for char in message:
7     encrypted.append(ord(char) ^ key)
8
9 print("\nEncrypted Data:")
10 print(encrypted)
11
12 decrypted = ""
13
14 for value in encrypted:
15     decrypted += chr(value ^ key)
16
17 print("\nDecrypted Message:")
18 print(decrypted)
```

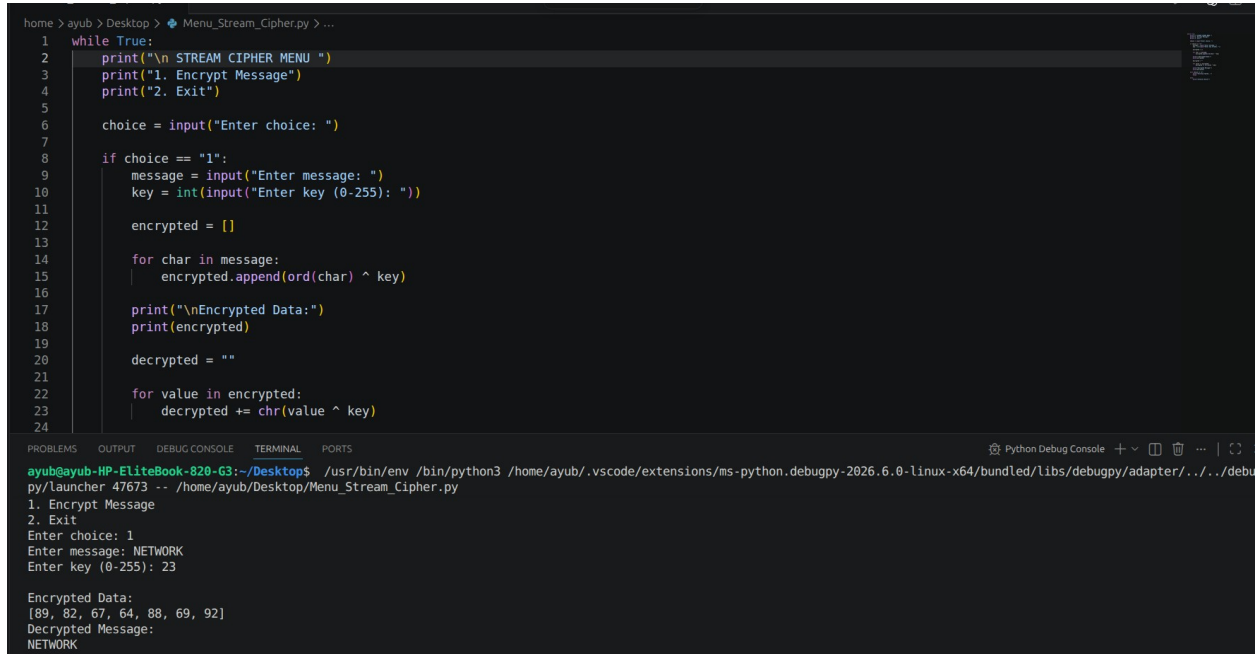
```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/Libs/debugpy/adapter/../../debugpy/launcher 60469 -- /home/ayub/Desktop/XOR_Modified.py
Enter message: My Name Jeff
Enter key (0-255): 55

Encrypted Data:
[122, 78, 23, 121, 86, 90, 82, 23, 125, 82, 81, 81]

Decrypted Message:
My Name Jeff
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 19: Modified XOR Cipher

Task 2



```
home > ayub > Desktop > Menu_Stream_Cipher.py > ...
1 while True:
2     print("\n STREAM CIPHER MENU ")
3     print("1. Encrypt Message")
4     print("2. Exit")
5
6     choice = input("Enter choice: ")
7
8     if choice == "1":
9         message = input("Enter message: ")
10        key = int(input("Enter key (0-255): "))
11
12        encrypted = []
13
14        for char in message:
15            encrypted.append(ord(char) ^ key)
16
17        print("\nEncrypted Data:")
18        print(encrypted)
19
20        decrypted = ""
21
22        for value in encrypted:
23            decrypted += chr(value ^ key)
24
25        print("\nDecrypted Message:")
26        print(decrypted)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debu
py/launcher 47673 -- /home/ayub/Desktop/Menu_Stream_Cipher.py

1. Encrypt Message
2. Exit
Enter choice: 1
Enter message: NETWORK
Enter key (0-255): 23

Encrypted Data:
[89, 82, 67, 64, 88, 69, 92]
Decrypted Message:
NETWORK

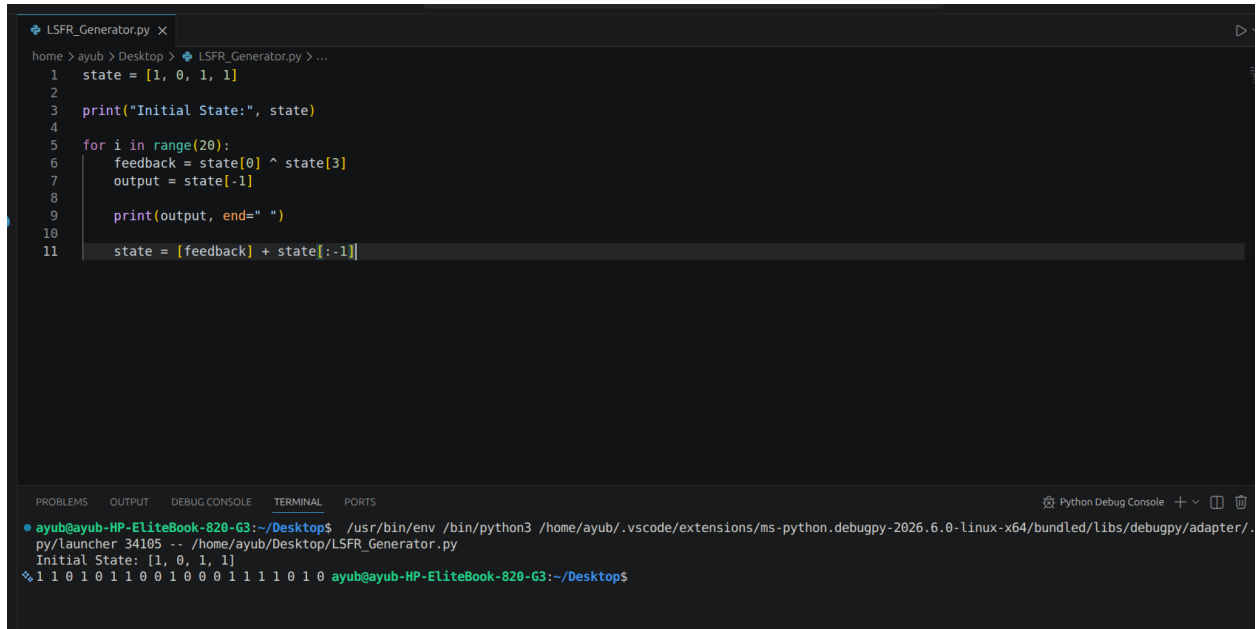
Figure 20: Menu Driven Stream Cipher Application

Task 3

	True Random Generators	Pseudorandom Generators
Security	Higher security. Values come from physical randomness.	Lower security if algorithm or seed becomes known.
Speed	Generally slower due to physical measurement processes.	Very fast, suitable for large scale applications.
Cost	More expensive because special hardware is often required.	Cheaper because only software algorithms are needed.
Practicality	Less practical for everyday computing.	Highly practical ,widely used in software systems.

Week 3: Stream Ciphers and Randomness Testing

Implementation of Stream Cipher Systems and Random Sequence Analysis

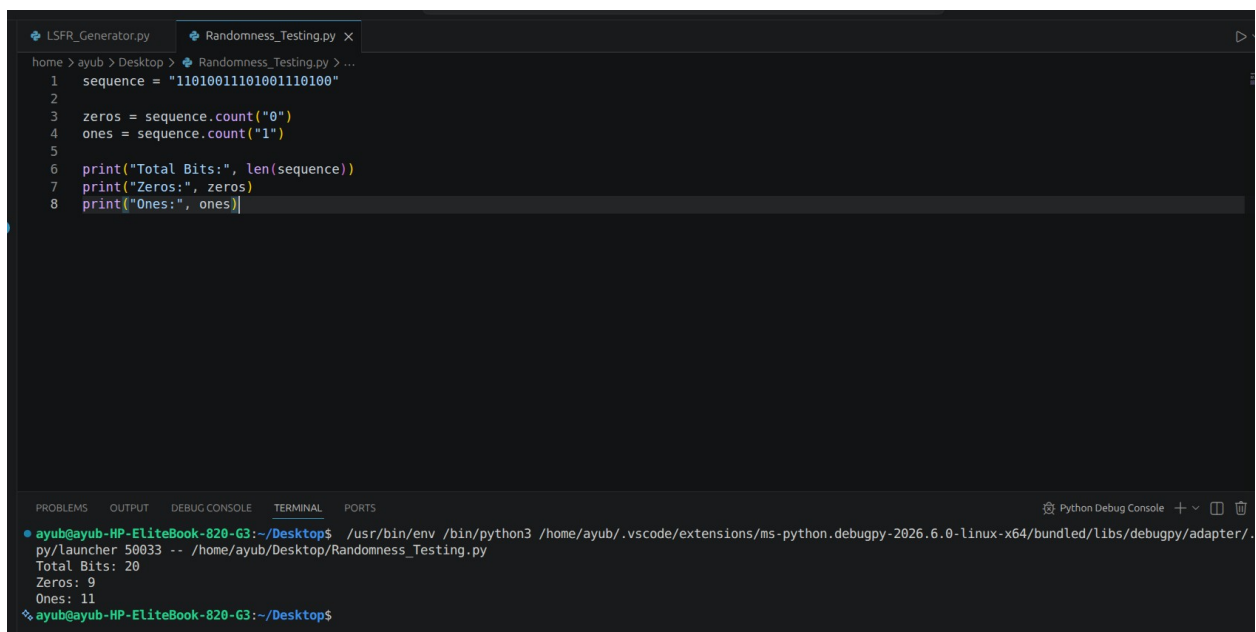


```
LSFR_Generator.py x
home > ayub > Desktop > LSFR_Generator.py > ...
1 state = [1, 0, 1, 1]
2
3 print("Initial State:", state)
4
5 for i in range(20):
6     feedback = state[0] ^ state[3]
7     output = state[-1]
8
9     print(output, end=" ")
10
11     state = [feedback] + state[:-1]
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [] [x]

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 34105 -- /home/ayub/Desktop/LSFR_Generator.py
Initial State: [1, 0, 1, 1]
1 1 0 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 1 0 ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 21: LFSR Generator Implementation

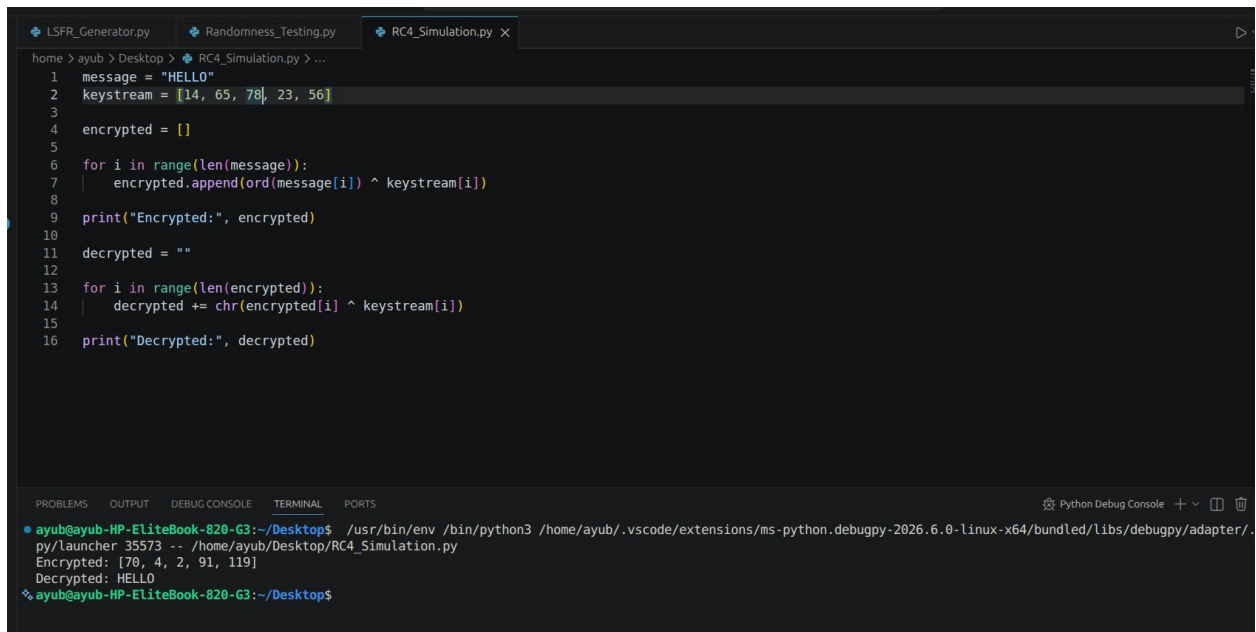


```
LSFR_Generator.py Randomness_Testing.py x
home > ayub > Desktop > Randomness_Testing.py > ...
1 sequence = "11010011101001110100"
2
3 zeros = sequence.count("0")
4 ones = sequence.count("1")
5
6 print("Total Bits:", len(sequence))
7 print("Zeros:", zeros)
8 print("Ones:", ones)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [] [x]

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 50033 -- /home/ayub/Desktop/Randomness_Testing.py
Total Bits: 20
Zeros: 9
Ones: 11
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 22: Statistical Randomness Testing



```
1 message = "HELLO"
2 keystream = [14, 65, 78, 23, 56]
3
4 encrypted = []
5
6 for i in range(len(message)):
7     encrypted.append(ord(message[i]) ^ keystream[i])
8
9 print("Encrypted:", encrypted)
10
11 decrypted = ""
12
13 for i in range(len(encrypted)):
14     decrypted += chr(encrypted[i] ^ keystream[i])
15
16 print("Decrypted:", decrypted)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + -

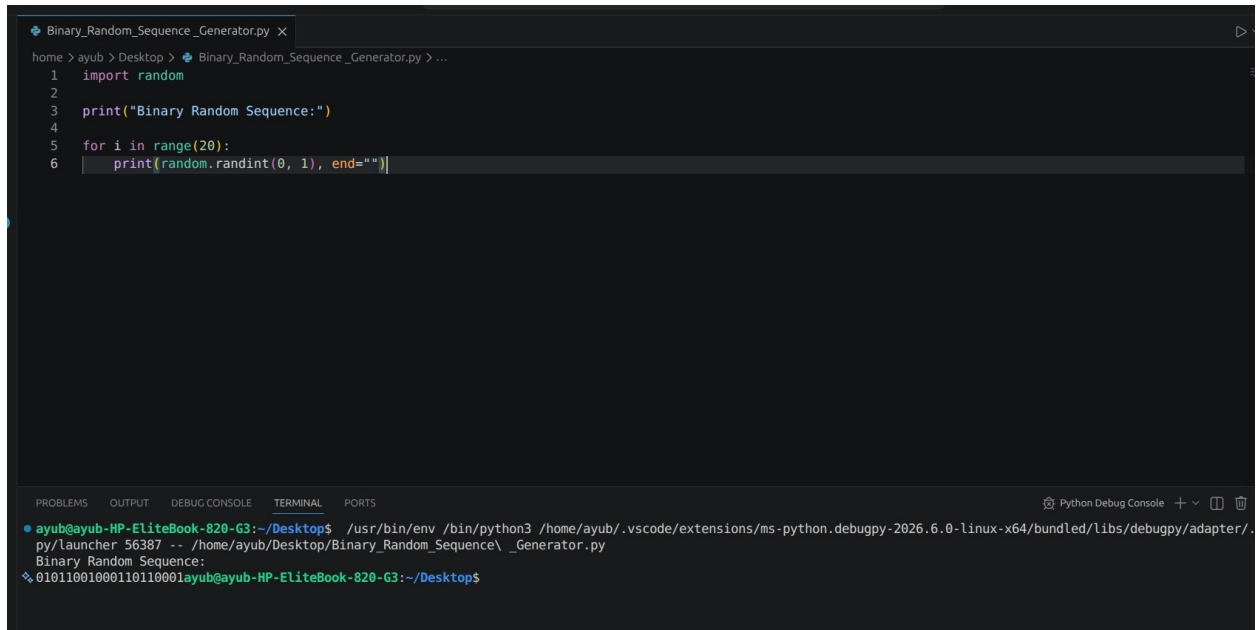
```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 35573 -- /home/ayub/Desktop/RC4_Simulation.py
Encrypted: [70, 4, 2, 91, 119]
Decrypted: HELLO
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 23: RC4 Stream Cipher Simulation

Reflection

I learned that randomness is essential in cryptography to ensure unpredictability of encryption keys. Stream ciphers operate by generating a keystream that is combined with plaintext. I performed basic statistical testing to check bit distribution. Pseudorandom generation is important for creating efficient yet secure encryption systems in practical applications.

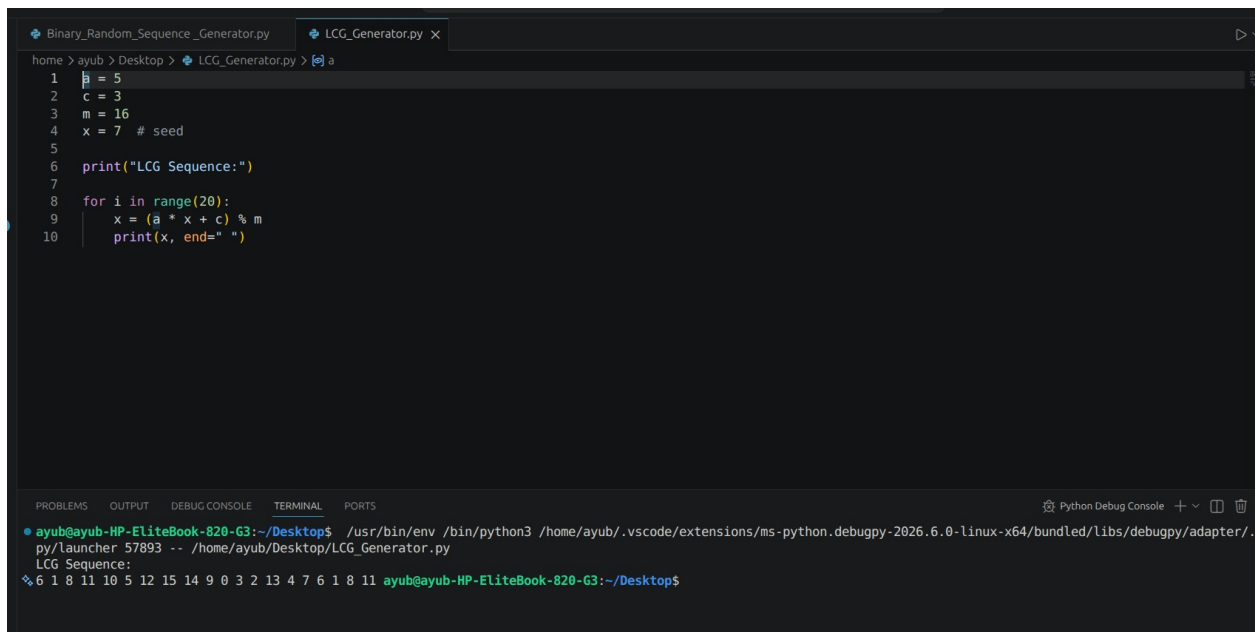
Practical Demonstrations



```
Binary_Random_Sequence_Generator.py x
home > ayub > Desktop > Binary_Random_Sequence_Generator.py > ...
1 import random
2
3 print("Binary Random Sequence:")
4
5 for i in range(20):
6     print(random.randint(0, 1), end=" ")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 56387 -- /home/ayub/Desktop/Binary_Random_Sequence/_Generator.py
Binary Random Sequence:
01011001000110110001ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

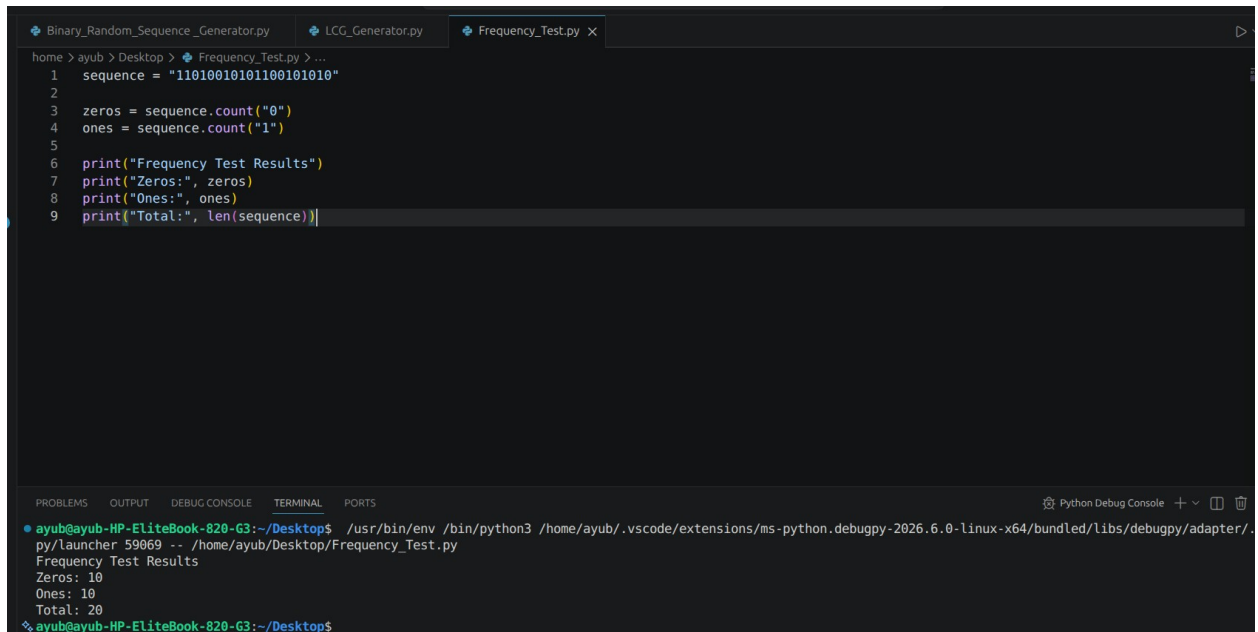
Figure 24: Generate binary random sequences



```
Binary_Random_Sequence_Generator.py LCG_Generator.py x
home > ayub > Desktop > LCG_Generator.py > a
1 a = 5
2 c = 3
3 m = 16
4 x = 7 # seed
5
6 print("LCG Sequence:")
7
8 for i in range(20):
9     x = (a * x + c) % m
10    print(x, end=" ")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 57893 -- /home/ayub/Desktop/LCG_Generator.py
LCG Sequence:
6 1 8 11 10 5 12 15 14 9 0 3 2 13 4 7 6 1 8 11ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 25: Implement an LCG generator.



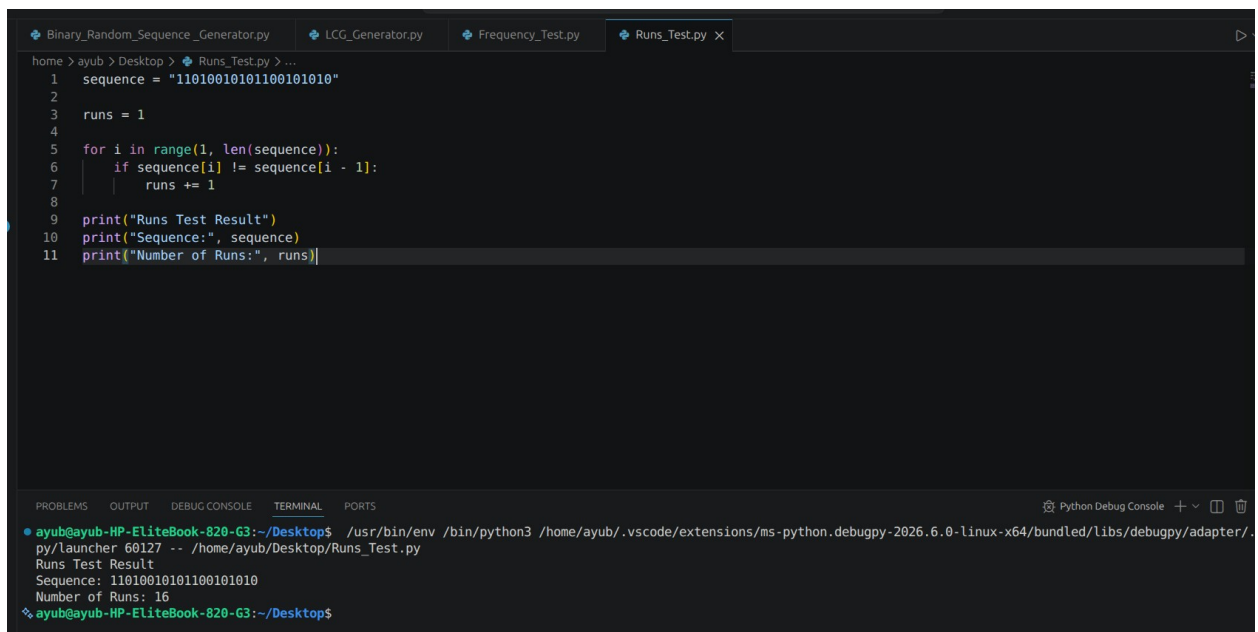
The screenshot shows a VS Code editor window with three tabs: Binary_Random_Sequence_Generator.py, LCG_Generator.py, and Frequency_Test.py. The Frequency_Test.py tab is active, displaying the following Python code:

```
1 sequence = "11010010101100101010"
2
3 zeros = sequence.count("0")
4 ones = sequence.count("1")
5
6 print("Frequency Test Results")
7 print("Zeros:", zeros)
8 print("Ones:", ones)
9 print("Total:", len(sequence))
```

The terminal at the bottom shows the execution of the script:

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 59069 -- /home/ayub/Desktop/Frequency_Test.py
Frequency Test Results
Zeros: 10
Ones: 10
Total: 20
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 26: Frequency Test



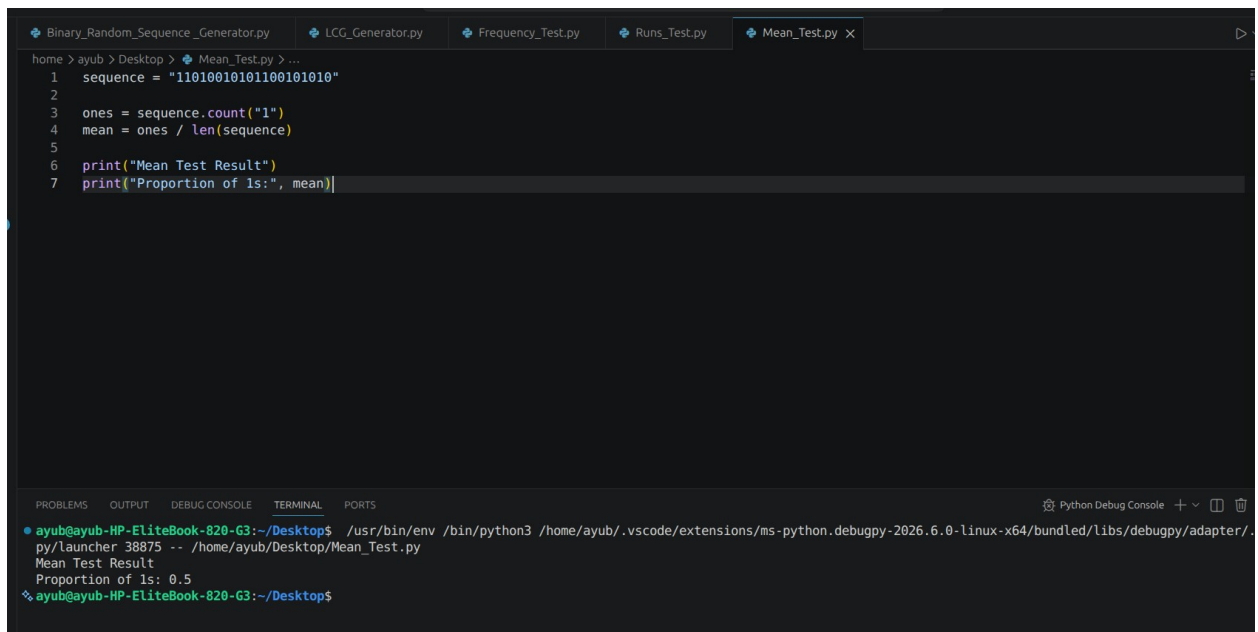
The screenshot shows a VS Code editor window with four tabs: Binary_Random_Sequence_Generator.py, LCG_Generator.py, Frequency_Test.py, and Runs_Test.py. The Runs_Test.py tab is active, displaying the following Python code:

```
1 sequence = "11010010101100101010"
2
3 runs = 1
4
5 for i in range(1, len(sequence)):
6     if sequence[i] != sequence[i - 1]:
7         runs += 1
8
9 print("Runs Test Result")
10 print("Sequence:", sequence)
11 print("Number of Runs:", runs)
```

The terminal at the bottom shows the execution of the script:

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 60127 -- /home/ayub/Desktop/Runs_Test.py
Runs Test Result
Sequence: 11010010101100101010
Number of Runs: 16
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 27: Runs Test



The image shows a VS Code editor window with a dark theme. At the top, there are five tabs: `Binary_Random_Sequence_Generator.py`, `LCG_Generator.py`, `Frequency_Test.py`, `Runs_Test.py`, and `Mean_Test.py` (which is the active tab). The editor displays the following Python code:

```
1 sequence = "11010010101100101010"
2
3 ones = sequence.count("1")
4 mean = ones / len(sequence)
5
6 print("Mean Test Result")
7 print("Proportion of 1s:", mean)
```

Below the editor, the `TERMINAL` panel is open, showing the command used to run the script and its output:

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/.py/launcher 38875 -- /home/ayub/Desktop/Mean_Test.py
Mean Test Result
Proportion of 1s: 0.5
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

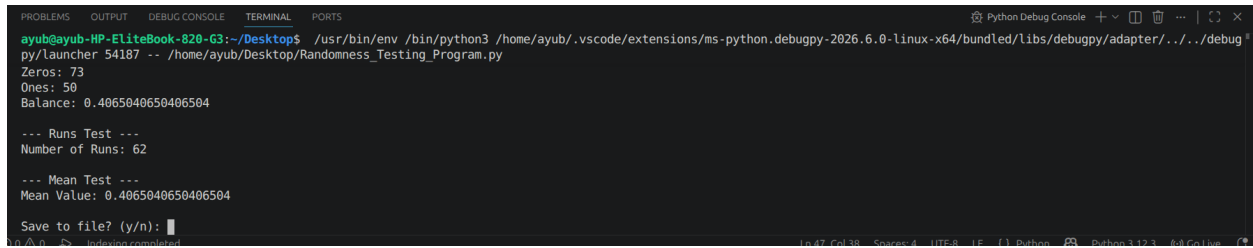
Figure 28: Mean Test

Assignment 1

```
Binary_Random_Sequence_Generator.py  LCG_Generator.py  Frequency_Test.py  Mean_Test.py  Randomness_Testing_Program.py X
home > ayub > Desktop > Randomness_Testing_Program.py > main
1  import random
2
3  def generate_bits(n):
4      return [random.randint(0, 1) for _ in range(n)]
5
6  def frequency_test(bits):
7      zeros = bits.count(0)
8      ones = bits.count(1)
9      total = len(bits)
10
11     print("\n--- Frequency Test ---")
12     print("Zeros:", zeros)
13     print("Ones:", ones)
14     print("Balance:", ones / total)
15
16     def runs_test(bits):
17         runs = 1
18
19         for i in range(1, len(bits)):
20             if bits[i] != bits[i - 1]:
21                 runs += 1
22
23         print("\n--- Runs Test ---")
24         print("Number of Runs:", runs)
25
26     def mean_test(bits):
27         mean = sum(bits) / len(bits)
28
29         print("\n--- Mean Test ---")
30         print("Mean Value:", mean)
31
32     def save_to_file(bits):
33         with open("random_bits.txt", "w") as f:
34             f.write("\n".join(map(str, bits)))
35         print("\nBits saved to random_bits.txt")
36
37     def analyze(bits):
38         print("\nGenerated Bits:")
39         print("\n".join(map(str, bits)))
40         frequency_test(bits)
41         runs_test(bits)
42         mean_test(bits)
43         save_to_file(bits)
44
45     def main():
46         while True:
47             print("\nRANDOMNESS TEST MENU")
48             print("1. Generate Random Bits")
49             print("2. Exit")
50
51             choice = input("Enter choice: ")
52
53             if choice == "1":
54                 n = int(input("Enter number of bits (100+ recommended): "))
55                 bits = generate_bits(n)
56
57                 analyze(bits)
58
59                 save = input("\nSave to file? (y/n): ")
60                 if save.lower() == "y":
61                     save_to_file(bits)
62
63             elif choice == "2":
64                 print("Exiting...")
65                 break
66
67             else:
68                 print("Invalid choice!")
69
70         main()
71
72 if __name__ == "__main__":
73     main()
```

```
Binary_Random_Sequence_Generator.py  LCG_Generator.py  Frequency_Test.py  Mean_Test.py  Randomness_Testing_Program.py X
home > ayub > Desktop > Randomness_Testing_Program.py > main
36     print("\nGenerated Bits:")
37     print("\n".join(map(str, bits)))
38
39     frequency_test(bits)
40     runs_test(bits)
41     mean_test(bits)
42     save_to_file(bits)
43
44     def main():
45         while True:
46             print("\nRANDOMNESS TEST MENU")
47             print("1. Generate Random Bits")
48             print("2. Exit")
49
50             choice = input("Enter choice: ")
51
52             if choice == "1":
53                 n = int(input("Enter number of bits (100+ recommended): "))
54                 bits = generate_bits(n)
55
56                 analyze(bits)
57
58                 save = input("\nSave to file? (y/n): ")
59                 if save.lower() == "y":
60                     save_to_file(bits)
61
62             elif choice == "2":
63                 print("Exiting...")
64                 break
65
66             else:
67                 print("Invalid choice!")
68
69         main()
70
71 if __name__ == "__main__":
72     main()
```

Figure 29: Randomness Testing Program



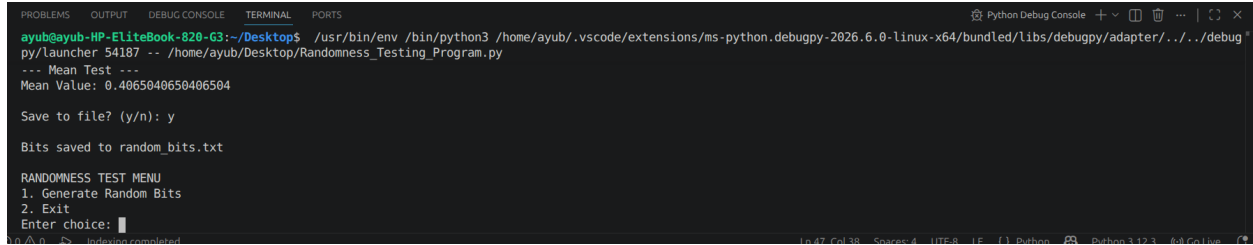
```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 54187 -- /home/ayub/Desktop/Randomness_Testing_Program.py
Zeros: 73
Ones: 50
Balance: 0.4065040650406504

--- Runs Test ---
Number of Runs: 62

--- Mean Test ---
Mean Value: 0.4065040650406504

Save to file? (y/n):
```

Figure 30: Statistical Tests



```
--- Mean Test ---
Mean Value: 0.4065040650406504

Save to file? (y/n): y

Bits saved to random_bits.txt

RANDOMNESS TEST MENU
1. Generate Random Bits
2. Exit
Enter choice:
```

Figure 31: Save to file

Assignment 2

Randomness in Cryptography Mini Research Report

Importance Of Randomness

Randomness is essential in cryptography because it ensures unpredictability in encryption keys and prevents attackers from guessing patterns. Strong randomness improves security in key generation, encryption, and authentication systems.

Weak Random Generators

Weak random generators produce predictable outputs if the seed or algorithm is known. Examples include simple linear congruential generators with poor parameters. These are vulnerable to attacks because their patterns can be reconstructed.

Historical Cryptographic Failures

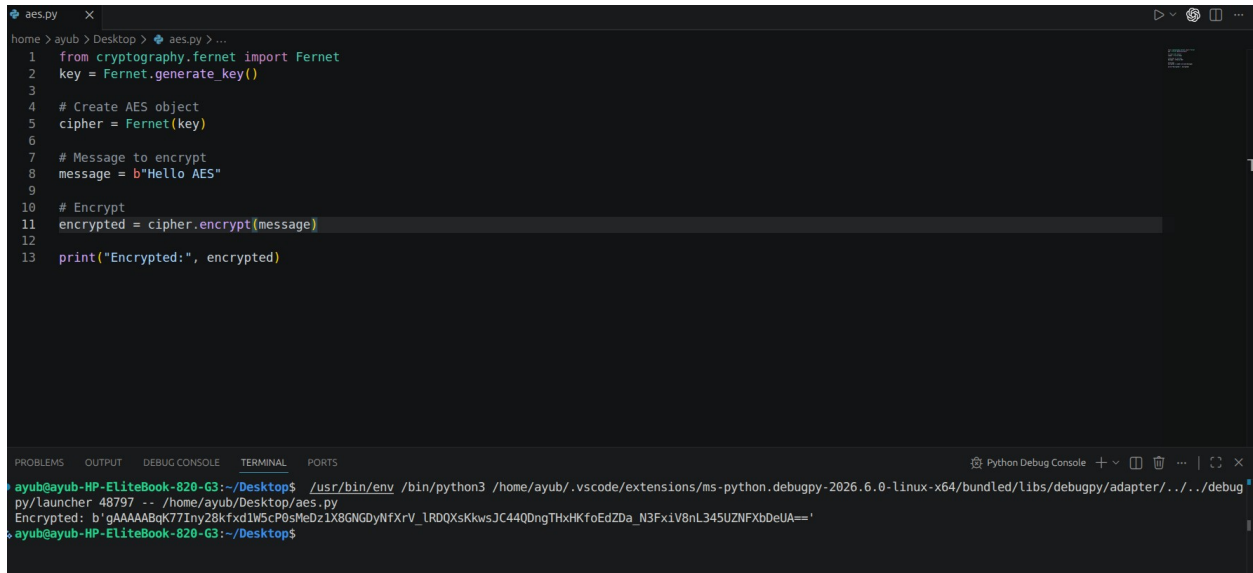
One major failure was the Debian OpenSSL bug, where a faulty random number generator produced predictable keys. Another issue is weak encryption keys caused

by poor randomness in early systems, leading to compromised secure communications.

Conclusion

Strong randomness is critical for secure cryptographic systems. Weak generators can lead to predictable encryption and serious security vulnerabilities. Modern systems rely on secure pseudorandom and hardware-based random generators.

Week 4: STREAM CIPHER CRYPTANALYSIS



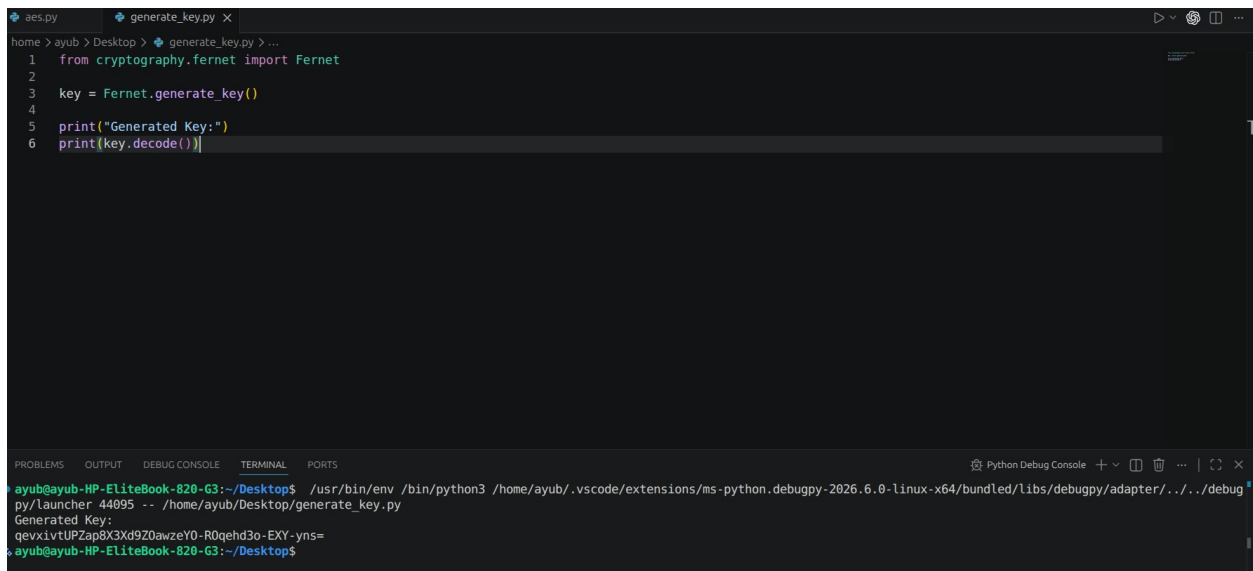
```
1 from cryptography.fernet import Fernet
2 key = Fernet.generate_key()
3
4 # Create AES object
5 cipher = Fernet(key)
6
7 # Message to encrypt
8 message = b"Hello AES"
9
10 # Encrypt
11 encrypted = cipher.encrypt(message)
12
13 print("Encrypted:", encrypted)
```

ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 48797 -- /home/ayub/Desktop/aes.py

Encrypted: b'gAAAAABgK77Iny28kfxdlW5cP0sMeDz1X8GNGDyNFXrV_LR0QXsKkwsJC44QDngTHxHKfoEdZDa_N3Fx1V8nL345UZNFXbDeUA=='

ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$

Figure 32: AES Encryption Script



```
1 from cryptography.fernet import Fernet
2
3 key = Fernet.generate_key()
4
5 print("Generated Key:")
6 print[key.decode()]
```

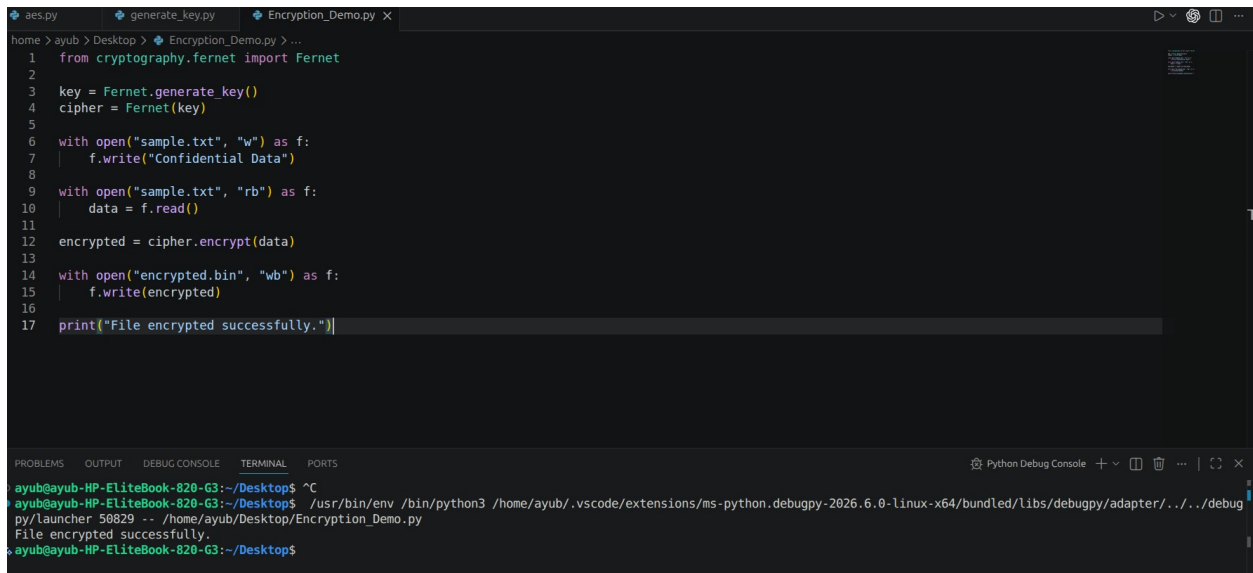
ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 44895 -- /home/ayub/Desktop/generate_key.py

Generated Key:

qevxiVtUPZap8X3Xd9Z0awzeY0-R0qehd3o-EXY-yns=

ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$

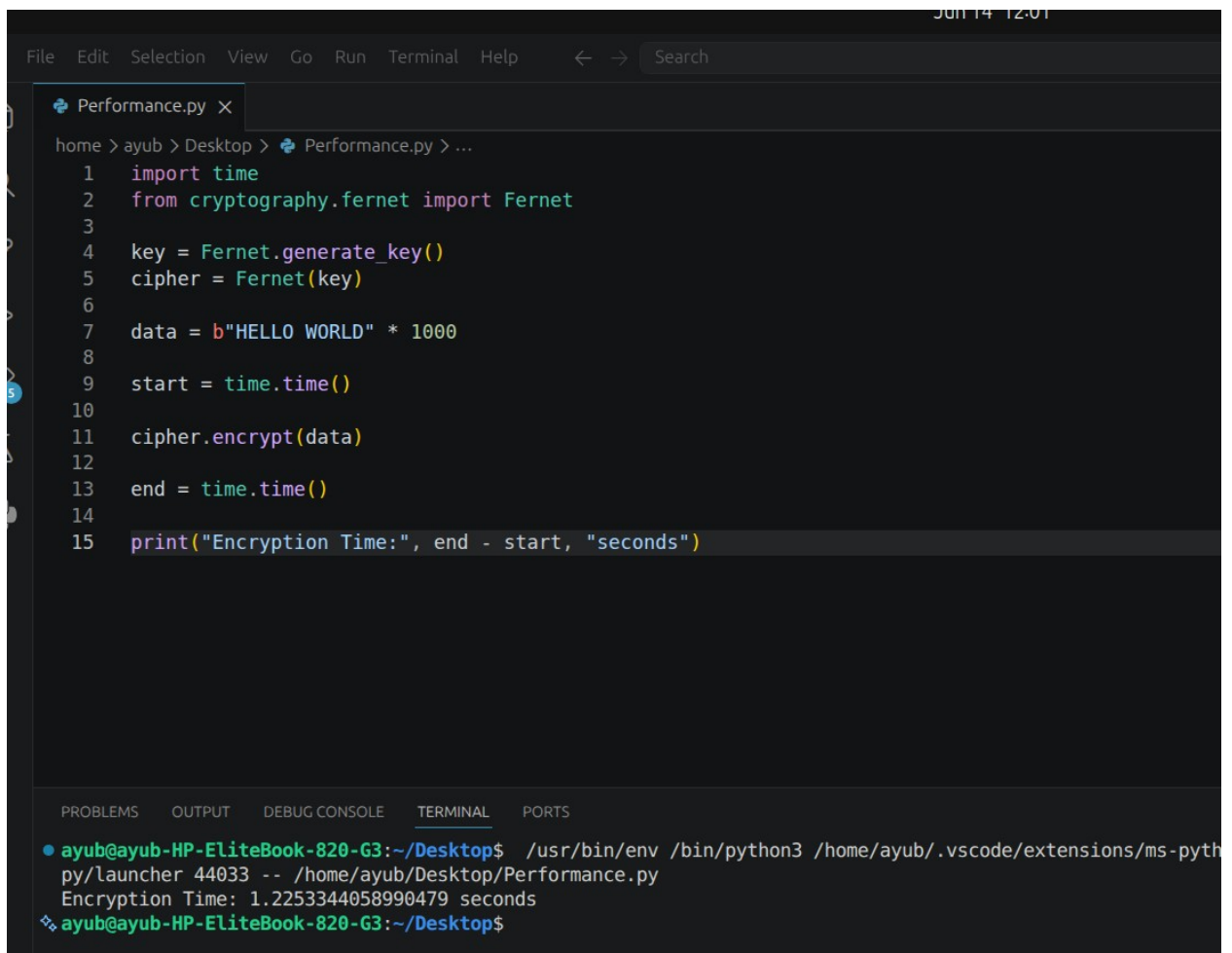
Figure 33: Key Generation Process



```
1 from cryptography.fernet import Fernet
2
3 key = Fernet.generate_key()
4 cipher = Fernet(key)
5
6 with open("sample.txt", "w") as f:
7     f.write("Confidential Data")
8
9 with open("sample.txt", "rb") as f:
10    data = f.read()
11
12 encrypted = cipher.encrypt(data)
13
14 with open("encrypted.bin", "wb") as f:
15    f.write(encrypted)
16
17 print("File encrypted successfully.")
```

ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$ ^C
ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 50829 -- /home/ayub/Desktop/Encryption_Demo.py
File encrypted successfully.
ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$

Figure 34: File Encryption Process



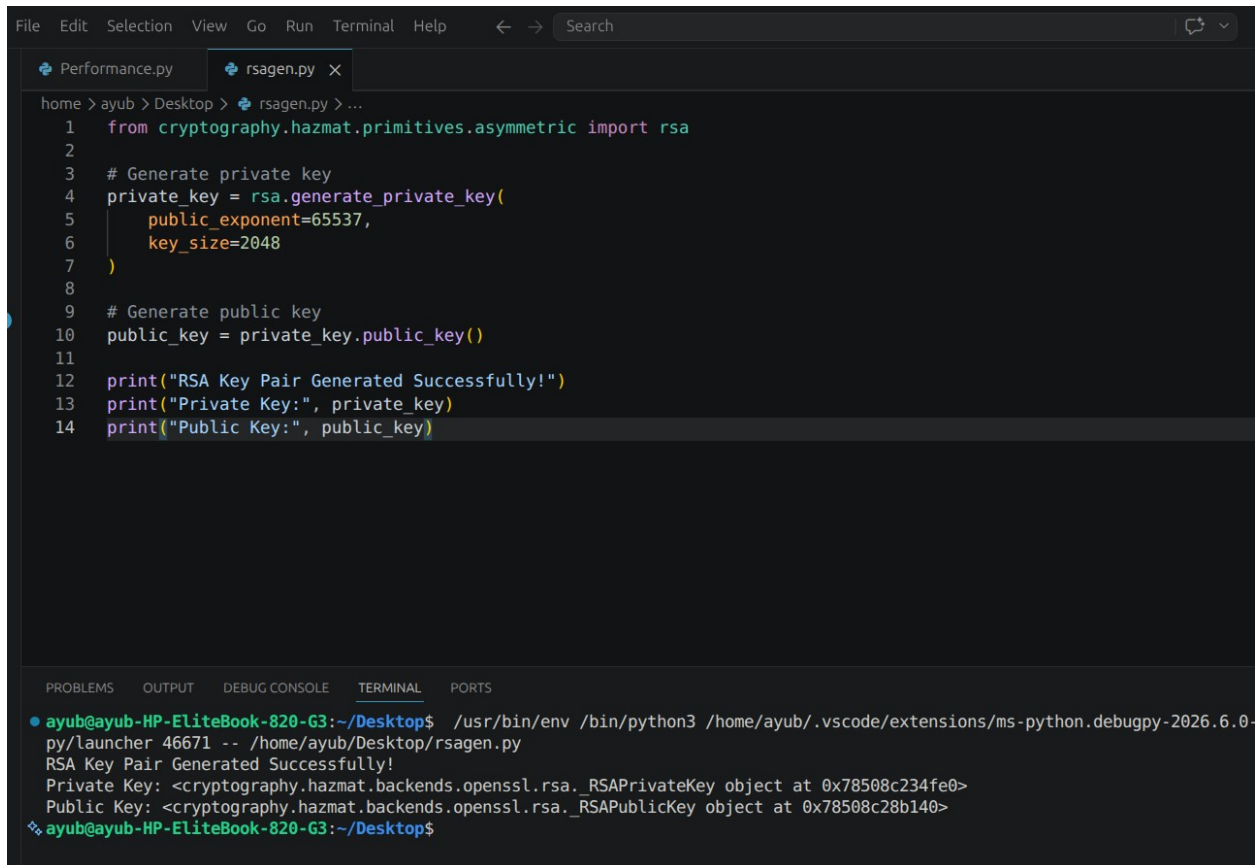
```
1 import time
2 from cryptography.fernet import Fernet
3
4 key = Fernet.generate_key()
5 cipher = Fernet(key)
6
7 data = b"HELLO WORLD" * 1000
8
9 start = time.time()
10
11 cipher.encrypt(data)
12
13 end = time.time()
14
15 print("Encryption Time:", end - start, "seconds")
```

ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/launcher 44033 -- /home/ayub/Desktop/Performance.py
Encryption Time: 1.2253344058990479 seconds
ayub@ayub-HP-EliteBook-820-G3:~/Desktop\$

Reflection

I learned that AES secures data by encrypting and decrypting information using secret keys. Key security is essential because anyone with the key can access the data. Implementing file encryption demonstrated the complete AES workflow, while managing keys and library setup were the main challenges encountered.

Week 5: Public Key Cryptography

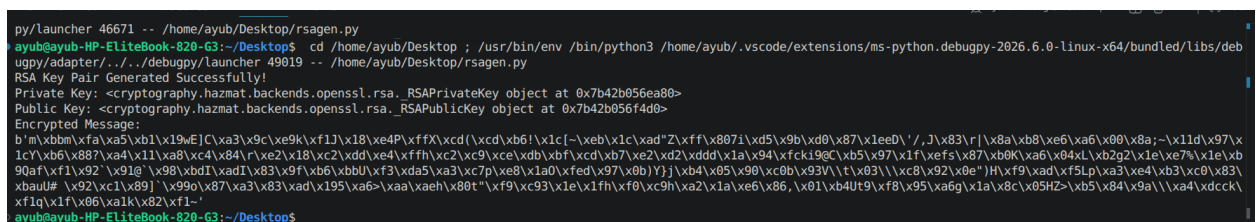


```
File Edit Selection View Go Run Terminal Help
Performance.py rsagen.py x
home > ayub > Desktop > rsagen.py > ...
1 from cryptography.hazmat.primitives.asymmetric import rsa
2
3 # Generate private key
4 private_key = rsa.generate_private_key(
5     public_exponent=65537,
6     key_size=2048
7 )
8
9 # Generate public key
10 public_key = private_key.public_key()
11
12 print("RSA Key Pair Generated Successfully!")
13 print("Private Key:", private_key)
14 print("Public Key:", public_key)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-py/launcher 46671 -- /home/ayub/Desktop/rsagen.py
RSA Key Pair Generated Successfully!
Private Key: <cryptography.hazmat.backends.openssl.rsa.RSAPrivateKey object at 0x78508c234fe0>
Public Key: <cryptography.hazmat.backends.openssl.rsa.RSAPublicKey object at 0x78508c28b140>
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 35: RSA Key Pair Generation



```
py/launcher 46671 -- /home/ayub/Desktop/rsagen.py
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ cd /home/ayub/Desktop ; /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/deb
ugpy/adapters/.../debugpy/launcher 49019 -- /home/ayub/Desktop/rsagen.py
RSA Key Pair Generated Successfully!
Private Key: <cryptography.hazmat.backends.openssl.rsa.RSAPrivateKey object at 0x7b42b056ea80>
Public Key: <cryptography.hazmat.backends.openssl.rsa.RSAPublicKey object at 0x7b42b056f4d0>
Encrypted Message:
b'm\xb6\xfa\x05\x11\x19\xe1c\xa3\x9c\xe9k\x11j\x18\xe4P\xffX\xcd(\xcd\xb6\x1c[-\xeb\x1c\xad"Z\xff\x87i\xd5\x9b\xd0\x87\x1ee0\'/J\x83\r\x8a\xb8\xe6\xa6\x00\x8a;- \x11d\x97\x
1cY\xb6\x887\x04\x11\xa8\x04\x84r\xe2\x18\x02\xdd\xe4\xffh\xc2\x09\xce\xdb\xbf\xcd\xb7\xe2\x02\xdd\x1a\x94\xfc\x198C\xb5\x97\x1f\xef\x87\xb0K\xa6\x04\x1b2g2\x1e\xe7\x1e\x
90af\x11\x02 \x91g\x98\xbdI\xadI\x87\x9f\xb6\xbbu\x1f2\xda5\xa3\xc7p\xe8\x1a0\xfed\x97\x0b)Yj\x04\x05\x90\x0b\x93V\t\x03\\\xc8\x92\x8e")H\x9f9\xad\x1fLp\xa3\xe4\xb3\x08\x83\
xbauI# \x92\x11\x89] \x99o\x87\xa3\x83\xad\x195\xa6>\xaa\xae\x80t"\xf9\xc93\x1e\x1fh\x0f\x0c9h\xa2\x1a\xe6\x86,\x01\xb4ut9\xf0\x95\xa6g\x1a\x8c\x05HZ>\xb5\x84\x9a\\\xa4\xdcck\
xf1q\x1f\x06\xa1k\x82\x1f1--'
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 36: Public Key Encryption Process



```
Decrypted Message:
Secure Communication
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 37: Private Key Decryption Results

```
51 if decrypted == message:
52     print("RSA Validation Successful!")
53 else:
54     print("RSA Validation Failed!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console + - [] ... | [] X

```
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$ cd /home/ayub/Desktop ; /usr/bin/env /bin/python3 /home/ayub/.vscode/extensions/ms-python.debugpy-2026.6.0-linux-x64/bundled/libs/debugpy/adapter/../../debugpy/launcher 59621 -- /home/ayub/Desktop/rsagen.py
bb\xd1\xab02\x87g\xel\xbf@:\x88yP\x85\x7f'
Decrypted Message:
Secure Communication

Secure Message Transmission
Original Message: Secure Communication
Encrypted Message Sent.
Private Key Used for Decryption.
Recovered Message: Secure Communication
RSA Validation Successful!
ayub@ayub-HP-EliteBook-820-G3:~/Desktop$
```

Figure 38: RSA Testing and Validation

Reflection

I learned that symmetric encryption uses one shared key, while asymmetric encryption uses separate public and private keys. RSA provides secure communication by protecting private keys and enabling public key distribution. Testing confirmed successful encryption, decryption, and validation of transmitted messages.